

DPDP COMPLIANCE AUDIT

saas-copilot

Digital Personal Data Protection Act 2023

64/100

Grade C

Needs Work

Scanned: 25 March 2026

Files indexed: 45

Languages: FastAPI, SQLAlchemy, Pydantic, Passlib, python-jose, uvicorn

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

The application collects user name, email, and password for authentication and authorization. The biggest DPDP risk is the storage and potential exposure of user credentials (password hash) and PII (email, name) without explicit consent mechanisms visible in the code.

Validation Summary Rule findings: 11 confirmed, 3 false positives suppressed • AI gaps: 2 dual-validated • Score: 65 (rules) → 64 (integrated)

Total findings: 13

HIGH: 4

MEDIUM: 3

INFO: 3

PASS: 1

Tech Stack: FastAPI, SQLAlchemy, Pydantic, Passlib, python-jose

Risk Profile: The application collects user name, email, and password for authentication and authorization. The biggest DPDP risk is the storage and potential exposure of user credentials (password hash) and PII (email, name) without explicit consent mechanisms visible in the code.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	11/22 (50%)	WARN
Section 6(6)	8	0/8 (0%)	FAIL
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	8/8 (100%)	PASS
Section 8(4)	8	0/8 (0%)	FAIL
Section 8(6)	8	6.8/8 (85%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	2.1/6 (35%)	FAIL
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 2.6–9.1 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: email, password

Login.jsx → api.js

Flow 2: Application Logs

PII fields: email, password

Signup.jsx → api.js

Rule	Section	Severity	File
CONSENT_MISSING_REPO_LEVEL	Section 6 — Consent	HIGH	REPO-WIDE
CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
DATA_ACCESS_MISSING	Section 11 — Right to Access	MEDIUM	N/A
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	backend/app/api/auth.py
	Section 9 — Children's Data	MEDIUM	
	Section 8(4) — Audit Trail	MEDIUM	

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (10–36 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Missing Repo Level — REPO-WIDE (4–16 hrs)
- Consent Withdrawal Missing (1–4 hrs)
- Audit Trail Missing (4–12 hrs)
- Grievance Officer Missing (1–4 hrs)

Week 2 — High Priority (6–22 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Data Access Missing (2–6 hrs)
- (2–8 hrs)
- (2–8 hrs)

Week 4 — Advisory (4–14 hours)

Address before next compliance review.

- No Breach Alerting (2–6 hrs)
- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 20–72 hours (2.5–9.0 engineering days)

Changes Since 2026-03-24

▼ **Score: 65.0 → 64 (-1.0 points)**

✓ 1 finding(s) resolved

■ 3 new finding(s) introduced

— 11 finding(s) unchanged

Resolved since last scan:

✓

New findings this scan:

- Password Not Hashed [HIGH]
- No Deletion Mechanism [MEDIUM]
- Retention Missing [MEDIUM]

Details

Finding 1 of 13

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING_REPO_LEVEL — CODEBASE-WIDE

Estimated fix time: 4–16 hours

No consent mechanism detected anywhere in the repository.

Risk:

The absence of a consent mechanism means that personal data processing activities may occur without the explicit agreement of data subjects, violating DPDP Section 6. This can lead to non-compliance penalties and a loss of user trust.

Fix:

1. Implement a consent management system, potentially as a new service or integrated into user profile management.
2. Add API endpoints to allow users to grant and revoke consent for specific data processing activities.
3. Modify data processing endpoints (e.g., in `rag.py` or other relevant services) to check for and respect user consent before processing personal data.
4. Update user interfaces to include clear consent requests and management options.

DPDP:

Section 6 — Consent requires data subjects to give consent for the processing of their personal data.

Example:

```
from fastapi import APIRouter, Depends from sqlalchemy.orm import Session from app.db.session import SessionLocal from app.models.consent import Consent router = APIRouter() def get_db(): db = SessionLocal() try: yield db finally: db.close() @router.post('/consent/{user_id}') def grant_consent(user_id: str, consent_data: dict, db: Session = Depends(get_db)): # Logic to record consent in the database pass @router.delete('/consent/{user_id}') def revoke_consent(user_id: str, db: Session = Depends(get_db)): # Logic to revoke consent in the database pass
```

Finding 2 of 13

[Section 6(6) — Consent Withdrawal]

2. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A:1

Evidence snippet: This frontend component manages user login and navigation but does not contain any logic for handling consent withdrawal.

Estimated fix time: 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

The absence of a consent withdrawal mechanism means users cannot easily revoke their consent, which is a direct violation of DPDP Section 6(6). This can lead to continued processing of data against a user's wishes, resulting in legal penalties and reputational damage.

Fix:

1. Implement an API endpoint in the backend (e.g., in `auth.py` or a dedicated consent management module) to handle consent revocation requests.
2. Create a corresponding UI element in the frontend (e.g., within user settings or profile page) that allows users to trigger the consent withdrawal.
3. Ensure the backend logic for consent withdrawal is as simple and accessible as the consent granting process.
4. Update user data models and database schemas to accurately reflect consent status changes.

DPDP:

Section 6(6) — Consent withdrawal must be as easy as giving consent, allowing users to revoke consent at any time.

Example:

```
from fastapi import APIRouter, Depends from sqlalchemy.orm import Session from app.db.session import SessionLocal from app.models.consent import Consent router = APIRouter() def get_db(): db = SessionLocal() try: yield db finally: db.close() @router.delete('/consent/{user_id}') def revoke_consent(user_id: str, db: Session = Depends(get_db)): # Find and remove or invalidate consent records for the user consent_record = db.query(Consent).filter(Consent.user_id == user_id).first() if consent_record: db.delete(consent_record) db.commit() return {"message": "Consent revoked successfully"} return {"message": "No consent found to revoke"}
```

Finding 3 of 13

[Section 8(4) — Audit Trail]

3. [HIGH] AUDIT_TRAIL_MISSING — N / A**Estimated fix time:** 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The absence of an audit trail means there are no records of personal data processing activities, violating DPDP Section 8(4). This hinders accountability, makes it difficult to investigate data breaches or misuse, and prevents demonstration of compliance.

Fix:

1. Define an audit log model (e.g., `AuditLog` with fields like `timestamp`, `user\_id`, `action`, `data\_accessed`, `details`).
2. Implement a logging mechanism that captures relevant events (e.g., data ingestion, data queries, user authentication) and stores them in the audit log.
3. Integrate audit logging into key functions and API endpoints, ensuring that actions involving personal data are recorded.
4. Consider using a dedicated logging library or service for more robust audit trail management.

DPDP:

Section 8(4) — Requires the Data Fiduciary to maintain complete and accurate records of all personal data processing activities.

Example:

```
from app.db.session import SessionLocal from app.models.audit_log import AuditLog from
datetime import datetime def log_audit_event(user_id: str, action: str, details: str = ""): db
= SessionLocal() try: log_entry = AuditLog( timestamp=datetime.utcnow(), user_id=user_id,
action=action, details=details ) db.add(log_entry) db.commit() finally: db.close() # Example
usage in an API endpoint: # log_audit_event(user_id='user123', action='data_ingest',
details='Ingested 10 documents')
```

Finding 4 of 13

[Section 13 — Grievance Redressal]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A**Estimated fix time:** 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route ****and**** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The absence of a designated grievance redressal officer means there is no clear point of contact for individuals to raise concerns or complaints regarding their personal data. This violates DPDP Section 13 and can lead to unresolved issues, increased user dissatisfaction, and potential legal challenges.

Fix:

1. Appoint a specific individual within the organization to act as the Grievance Redressal Officer.
2. Clearly document the contact information (e.g., email, phone number) for the Grievance Redressal Officer.
3. Publish the Grievance Redressal Officer's contact details on the company's privacy policy and website.
4. Establish a process for handling and responding to grievances received by the officer.

DPDP:

Section 13 — Requires the Data Fiduciary to appoint a Grievance Redressal Officer to address data principal's grievances.

Example:

This is a policy/procedural fix, not directly code-implementable within this snippet. #
However, you would update your privacy policy and potentially add a contact point. # Example
of how you might reference it in a contact page or privacy policy: # Privacy Policy Snippet: #
For any grievances or concerns regarding the processing of your personal data, # please
contact our Grievance Redressal Officer: # Name: [Officer's Name] # Email: [Officer's Email
Address] # Phone: [Officer's Phone Number]

Finding 5 of 13

[Section 11 — Right to Access]

5. [MEDIUM] DATA_ACCESS_MISSING — N / A**Estimated fix time:** 2–6 hours

No endpoint detected that allows authenticated users to retrieve their own personal data. Under DPDP Section 11, Data Principals have the right to access information about their personal data being processed.

Risk:

The absence of a data access endpoint means users cannot exercise their right to access their personal data, leading to non-compliance with DPDP Section 11.

Fix:

1. Create a new FastAPI endpoint (e.g., GET /users/me/data) that requires user authentication.
2. In the endpoint, retrieve the authenticated user's ID.
3. Query the database for all personal data associated with that user ID.
4. Return the data in a structured format (e.g., JSON).

DPDP:

Section 11 — Right to Access: Data Principals have the right to obtain confirmation that their personal data is being processed and to access such data.

Example:

```
from fastapi import APIRouter, Depends from app.database import get_db from app.models.user import User # Assuming a User model from app.schemas.user import UserDataSchema # Assuming a Pydantic schema from app.security import get_current_user router = APIRouter() @router.get('/users/me/data', response_model=UserDataSchema) def get_my_data(current_user: User = Depends(get_current_user)): # Fetch user data from DB using current_user.id user_data = fetch_user_data_from_db(current_user.id, db=get_db()) return user_data
```

Finding 6 of 13

[Section 8(6) — Breach]

6. [LOW] NO_BREACH_ALERTING — N / A**Estimated fix time:** 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

Without automated breach alerting, the organization cannot promptly notify affected individuals and the Authority as required by DPDP Section 8(6), increasing the potential harm from a data breach.

Fix:

1. Integrate a structured logging library (e.g., `python-json-logger`) to ensure logs are machine-readable.
2. Configure an error monitoring service (e.g., Sentry, Rollbar) to capture exceptions and critical events.
3. Set up alerts within the monitoring service to notify a designated compliance team or email address upon detection of potential breach indicators.
4. Define specific log events or error patterns that should trigger a breach alert.

DPDP:

Section 8(6) — Breach: Data Fiduciaries must notify the Authority and affected Data Principals without undue delay when a personal data breach occurs.

Example:

```
import logging from sentry_sdk import capture_exception logger = logging.getLogger(__name__)
def handle_potential_breach(user_id, data): try: # Log the incident with relevant details
logger.error(f"Potential breach detected for user {user_id}: {data}") # Send to Sentry for
alerting capture_exception(Exception(f"Potential breach: {user_id}")) # Additional
notification logic can be added here except Exception as e: logger.critical(f"Failed to log or
alert breach: {e}")
```

Finding 7 of 13

[Section 11 — Data Portability]

7. [LOW] DATA_PORTABILITY_MISSING — N / A**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The lack of a data export feature prevents users from obtaining their personal data in a usable format, failing to meet the requirements of DPDP Section 11 regarding data portability.

Fix:

1. Create a new FastAPI endpoint (e.g., GET /users/me/export) that requires user authentication.
2. Retrieve the authenticated user's data from the database.
3. Format the data into a common, machine-readable format such as CSV or JSON.
4. Return the formatted data as a file download.

DPDP:

Section 11 — Data Portability: Data Fiduciaries shall provide Data Principals with their personal data in a structured, commonly used, and machine-readable format.

Example:

```
from fastapi import APIRouter, Depends, Response
import pandas as pd
from io import StringIO
from app.security import get_current_user
from app.database import get_db

router = APIRouter()

@router.get('/users/me/export')
def export_my_data(current_user = Depends(get_current_user)):
    user_data = fetch_user_data_from_db(current_user.id, db=get_db())
    df = pd.DataFrame(user_data)
    csv_buffer = StringIO()
    df.to_csv(csv_buffer, index=False)
    return Response(
        content=csv_buffer.getvalue(),
        media_type='text/csv',
        headers={'Content-Disposition': 'attachment; filename="user_data.csv"'})
```

Finding 8 of 13

[Section 5 — Notice]

8. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 9 of 13

[Section 5 — Purpose Limitation]

9. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 10 of 13

[Section 16 — Cross-Border Transfer]

10. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 11 of 13

[Section 6 — Consent]

11. [PASS] CONSENT_PRESENT — backend / app / api / auth.py

Auth enforcement pattern detected inline — consent likely handled at registration.

Finding 12 of 13

[Section 9 — Children's Data]

12. [MEDIUM] — CODEBASE-WIDE

No age verification for new users

Finding 13 of 13

[Section 8(4) — Audit Trail]

13. [MEDIUM] — CODEBASE-WIDE

Missing audit trail for PII access

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] No age verification for new users

Confidence: 55%

Section 9 — Children's Data

The signup process (backend/app/api/auth.py) collects user name and email but does not include any fields or logic to verify the age of the user. This could lead to the collection of children's personal data without appropriate consent.

Recommendation: Modify the user signup schema (backend/app/schemas/user.py) to include an 'age' field and implement validation logic in the signup endpoint (backend/app/api/auth.py) to ensure users meet the minimum age requirement, or to trigger parental consent flows if applicable.

[AI-INFERRED] Missing audit trail for PII access

Confidence: 55%

Section 8(4) — Audit Trail

The data flows indicate that request method, URL, status code, duration, and trace ID are logged via HTTP middleware (backend/app/utils/logger.py). However, there is no explicit mention or evidence of logging specific access events to user personal data (e.g., viewing user profiles, accessing user-related documents) as required by Section 8(4).

Recommendation: Implement specific logging for all read operations on user personal data fields (id, name, email in models/user.py) and user-related data. This logging should capture who accessed the data, when, and what data was accessed, and should be stored securely.